

J2 – Design des endpoints

Branche `day2/endpoints-design`, à partir de `day1/spectral-rules`. Un seul `GET /products`. Le path est un nom. La page et les filtres sont des query params.

1. Naming

`docs/CHEATSHEET-NAMING.md` · `.spectral.yml`

On écrit	On n'écrit pas
<code>/products</code>	<code>/product</code> , <code>/getAllProducts</code>
<code>/order-items</code>	<code>/orderItems</code>
<code>DELETE /products/42</code>	<code>POST /products/delete/42</code>
<code>listProducts</code>	<code>list-products</code>

`path-kebab-case` et `operation-id-camel-case` sont en error. La CI bloque. Le path nomme la ressource, le verbe HTTP porte l'action.

```
spectral lint openapi/**/*.yaml --ruleset .spectral.yml
```

2. Les URLs

`openapi/products-v1.yaml`

Path	Verbes
<code>/products</code>	GET, POST
<code>/products/{id}</code>	GET, PUT, DELETE

Deux niveaux maximum : `/orders/{orderId}/items`. Un filtre n'est pas un nouveau path : `GET /products?categoryId=1`.

3. Entité Category

`src/main/java/tn/clevory/api/entity/Category.java`

```
@Entity
@Table(name = "categories")
public class Category {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true, length = 80)
    private String name;
    @Column(length = 255)
    private String description;
}
```

Remplace le faux nom `Category-1`. Hibernate crée la table (`ddl-auto: update`).

4. CategoryRepository

`src/main/java/tn/clevory/api/repository/CategoryRepository.java`

```
public interface CategoryRepository extends JpaRepository<Category, Long> {}
```

`findById` est déjà fourni. On n'écrit pas de SQL.

5. Données de démo

`src/main/resources/data.sql`

```
INSERT INTO categories (name, description) VALUES
('Smartphones', 'Téléphones mobiles'),
('Laptops', 'Ordinateurs portables'),
('Audio', 'Écouteurs et enceintes');
```

Avant les produits. Ids 1, 2, 3. iPhone et Galaxy → 1, MacBook et Dell → 2, AirPods → 3. Chargé par `application.yml`.

6. Le vrai nom de catégorie

src/main/java/tn/clevory/api/service/ProductService.java – create et update

```
categoryRepository.findById(request.getCategoryId())
    .ifPresent(c -> entity.setCategoryName(c.getName()));
```

À la place de "Category-" + id. categoryId: 1 donne Smartphones. Id inconnu : on ne plante pas.

7. Pagination

ProductController.java · dto/ProductPageDto.java

```
@PageableDefault(size = 20) Pageable pageable
```

URL	Sens	Défaut
page	numéro, commence à 0	0
size	taille de la page	20
sort	price,desc	aucun

Réponse : content, page, size, totalElements, totalPages. 5 produits et size=2 → totalPages: 3. Page suivante : page=1.

8. Filtres

ProductController.java – méthode list

```
@RequestParam(required = false) String q,
@RequestParam(required = false) Long categoryId,
@RequestParam(required = false) BigDecimal minPrice,
@RequestParam(required = false) BigDecimal maxPrice,
```

Param	Effet
q	le nom contient ce texte
categoryId	une seule catégorie
minPrice	price >= minPrice
maxPrice	price <= maxPrice

required = false, sinon GET /products seul répond 400. Les filtres se cumulent avec page, size et sort.

9. ProductRepository

src/main/java/tn/clevory/api/repository/ProductRepository.java

```
public interface ProductRepository
    extends JpaRepository<Product, Long>, JpaSpecificationExecutor<Product> {}
```

findAll(Specification, Pageable) : WHERE dynamique et page en même temps.

10. Un prédicat par filtre

src/main/java/tn/clevory/api/service/ProductSpecifications.java

```
public static Specification<Product> hasNameContaining(String q) {
    return (root, query, cb) -> {
        if (q == null || q.isBlank()) return cb.conjunction();
        return cb.like(cb.lower(root.get("name")), "%" + q.toLowerCase() + "%");
    };
}
```

Paramètre absent → conjunction() → ce filtre ne retire rien.

Méthode	SQL
hasCategoryId	category_id = ?
priceGreaterThanOrEqual	price >= ?
priceLessThanOrEqual	price <= ?

11. Assembler la page

ProductService.java – méthode list

```
Specification<Product> spec = Specification
    .where(ProductSpecifications.hasNameContaining(q))
    .and(ProductSpecifications.hasCategoryId(categoryId))
    .and(ProductSpecifications.priceGreaterThanOrEqual(minPrice))
    .and(ProductSpecifications.priceLessThanOrEqual(maxPrice));

Page<Product> page = repository.findAll(spec, pageable);

return new ProductPageDto(
    page.map(this::toDto).getContent(),
    page.getNumber(), page.getSize(),
    page.getTotalElements(), page.getTotalPages());
```

Le WHERE et le LIMIT sont en SQL (show-sql: true).

12. Vérifier

```
./mvnw spring-boot:run
```

Appel	Résultat
GET /products?size=2	2 produits, totalElements 5, totalPages 3
GET /products?categoryId=2	MacBook et Dell, categoryName Laptops
GET /products?q=pro	iPhone 16 Pro et MacBook Pro 14
GET /products?minPrice=1000&maxPrice=1500&sort=price,asc	Galaxy 1099, iPhone 1229, Dell 1499

Swagger : http://localhost:8080/swagger-ui.html